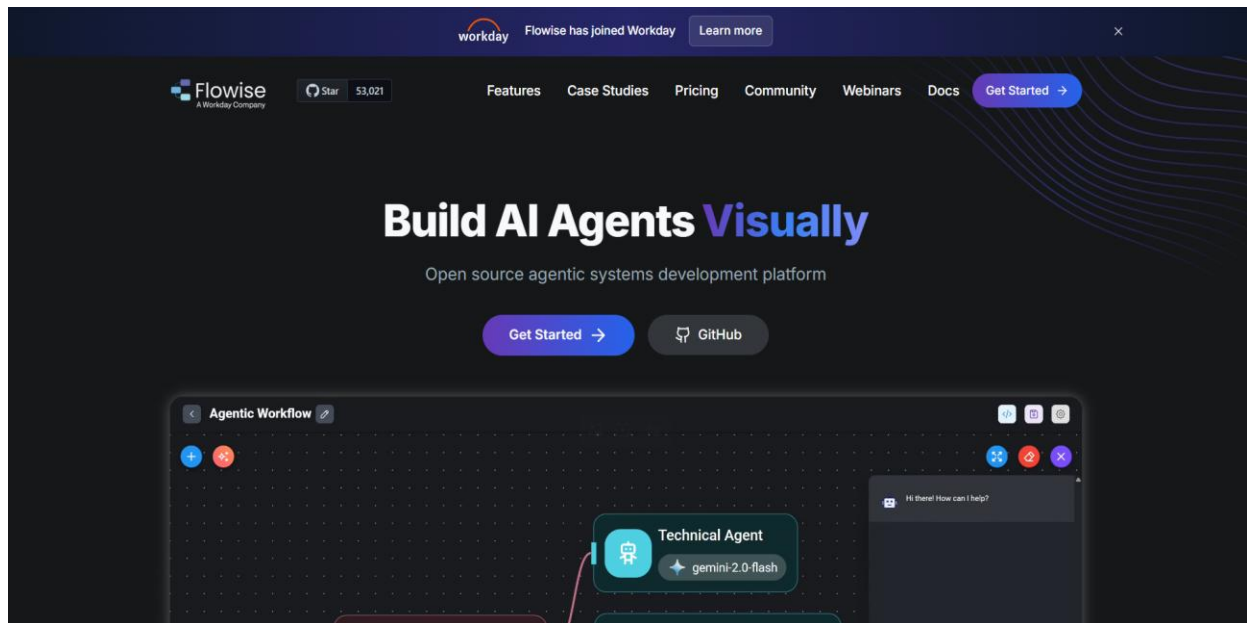

RAG Chatbot

Build a Retrieval-Augmented Generation chatbot that answers questions from your own documents — using Flowise AI, Google Gemini Embeddings, and Mistral AI.



Overview

A RAG (Retrieval-Augmented Generation) chatbot combines a language model with your own documents. Instead of relying on training data alone, the system retrieves relevant passages from your files and uses them as context when answering every question.

What You Will Build

- A Flowise chatflow that ingests a PDF document
 - A vector store that indexes document chunks via Gemini embeddings
 - A conversational QA chain powered by Mistral AI
 - A chatbot that remembers context across multiple conversation turns
-

Components at a Glance

Component	Tool	Purpose
Text Splitter	Recursive Character Text Splitter	Chunks PDF into overlapping segments
File Loader	PDF / File Loader	Loads and parses your PDF document
Embeddings	Google Gemini Embedding	Converts text chunks into vectors
Vector Store	In-Memory Vector Store	Stores & retrieves similar chunks
Memory	Buffer Memory	Keeps track of conversation history
Chat Model	Mistral AI (mistral-tiny)	Generates natural language answers
Chain	Conversational Retrieval QA Chain	Connects all components together

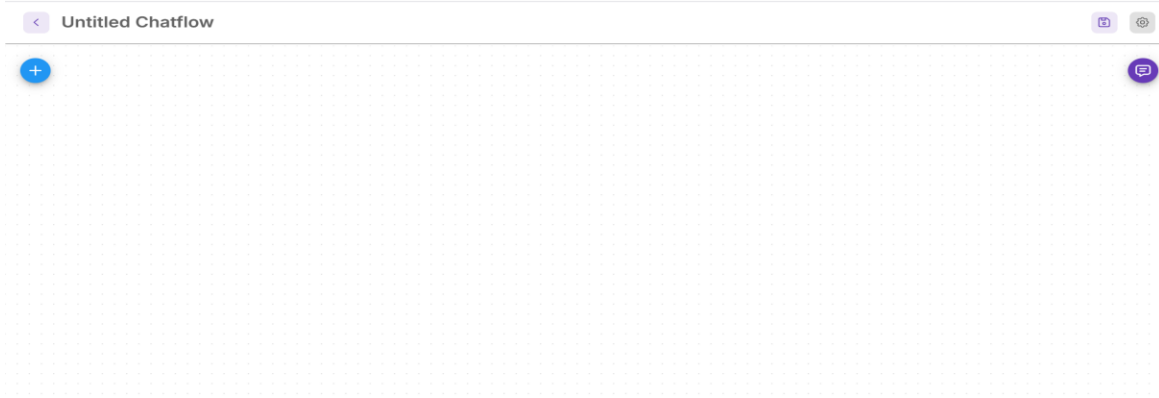
1 Sign Up for a Flowise AI Account

Visit flowiseai.com and create a free account. Flowise is an open-source, low-code platform for building LLM workflows visually.

2 Create a New Chatflow

- Open Flowise and navigate to the Chatflows dashboard
- Click “Add New” to create a blank flow
- Give the flow a name such as “RAG Chatbot”

You will see the empty canvas shown below:



3 Add Recursive Character Text Splitter

From the left sidebar, find and drag this node onto the canvas:

Text Splitters → Recursive Character Text Splitter

Configure with these settings:

Parameter / Setting	Value
Chunk Size	1000
Chunk Overlap	200

Tip — Chunk Size controls how large each text segment is. Chunk Overlap (200) ensures context is preserved across chunk boundaries, so the model can understand passages that span two adjacent chunks.

4 Add File Loader

From the left sidebar:

Document Loaders → File Loader

- Drag the node onto the canvas

- Click “Upload File” and upload your PDF (e.g. Cloud_Computing_Guide.pdf)

Connect: Text Splitter → File Loader

Draw a connection from the RecursiveCharacterTextSplitter output port to the Text Splitter input port on the File Loader node.

5 Add Google Gemini Embeddings

From the left sidebar:

Embeddings → Google Gemini Embedding

Configure with the following settings:

Parameter / Setting	Value
Credential	Gemini API Key
Model	gemini-embedding-001
Task Type	RETRIEVAL_DOCUMENT

Get your free API key from [Google AI Studio](#).

Tip — RETRIEVAL_DOCUMENT optimises the embedding for document storage. Use RETRIEVAL_QUERY when embedding user questions at query time.

6 Add In-Memory Vector Store

From the left sidebar:

Vector Stores → In-Memory Vector Store

Configure:

Parameter / Setting	Value
Top K	4

Tip — Top K = 4 means the system retrieves the 4 most relevant document chunks for each user question. Increase for broader context; decrease for more focused answers.

7 Add Buffer Memory

From the left sidebar:

Memory → Buffer Memory

- Drag it onto the canvas
- No additional configuration required for basic use

Tip — Buffer Memory stores the full conversation history in RAM. Each new message sees all previous turns, enabling follow-up questions like “Tell me more about that last point.”

8 Add Mistral AI Chat Model

From the left sidebar:

Chat Models → MistralAI

Configure with:

Parameter / Setting	Value
Credential	Your Mistral API Key
Model	mistral-tiny
Temperature	0.9

Get your API key from console.mistral.ai.

Tip — Temperature 0.9 gives varied, conversational responses. For factual Q&A over documents, lower to 0.3–0.5 for more precise, grounded answers.

9 Add Conversational Retrieval QA Chain

From the left sidebar:

Chains → Conversational Retrieval QA Chain

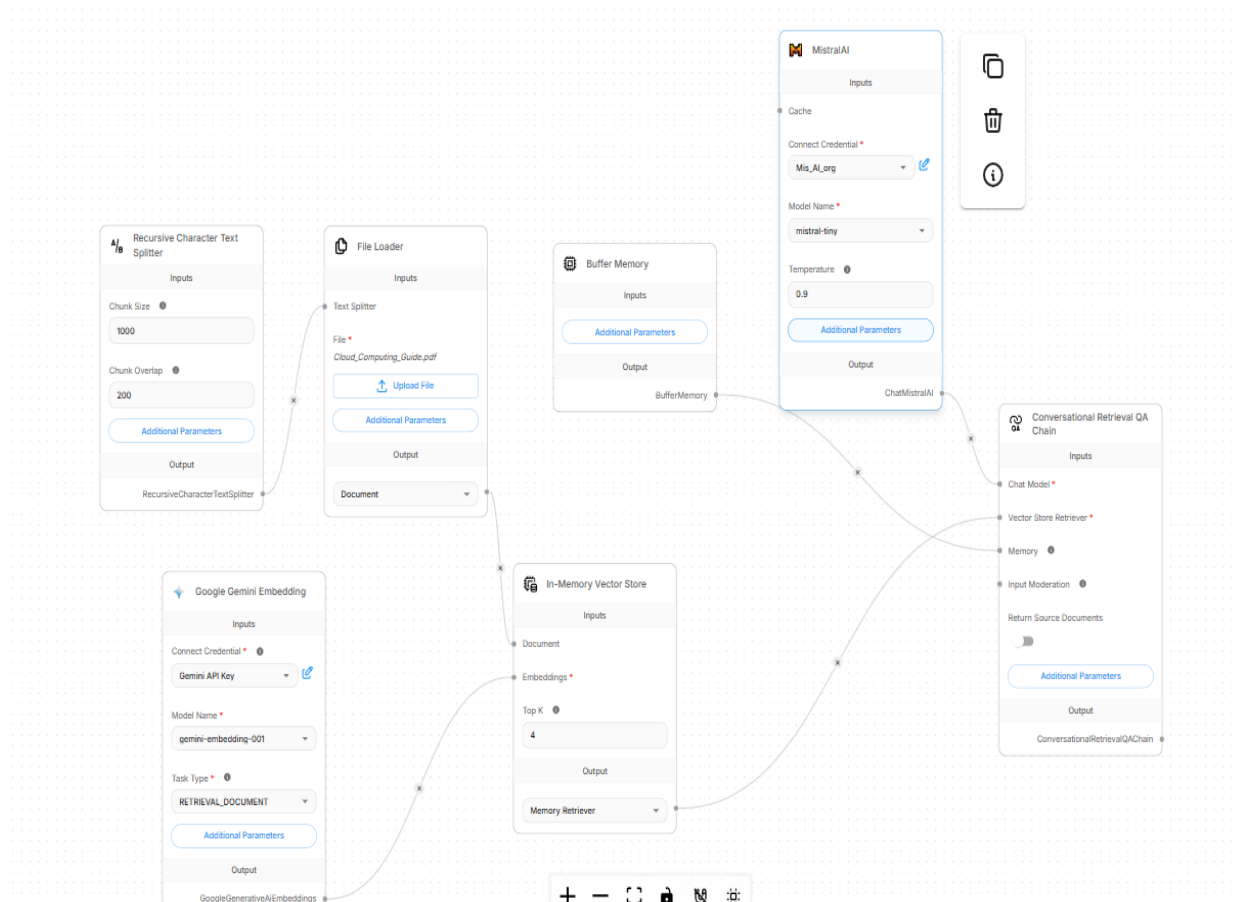
This is the brain of the system. It combines the retrieved document chunks with conversation history and sends everything to Mistral AI to produce contextual, grounded answers.

10 Connect Everything

Make the following connections on the canvas exactly as listed:

From Node	Output Port	To Node (Input Port)
Recursive Character Text Splitter	RecursiveCharacterTextSplitter	File Loader (Text Splitter port)
File Loader	Document	In-Memory Vector Store (Document port)
Google Gemini Embedding	GoogleGenerativeAIEmbeddings	In-Memory Vector Store (Embeddings port)
In-Memory Vector Store	Memory Retriever	Conv. Retrieval QA Chain (Vector Store Retriever)
Buffer Memory	BufferMemory	Conv. Retrieval QA Chain (Memory port)

Your completed canvas should look like the screenshot below:



How It All Works Together

When a user sends a message, the following pipeline runs automatically:

- 1 User types a question in the chat interface
- 2 Conversational Retrieval QA Chain receives the question + conversation history
- 3 The question is embedded with Gemini and compared against the vector store
- 4 Top 4 matching document chunks are retrieved from In-Memory Vector Store

-
- 5 Mistral AI receives: system prompt + retrieved chunks + history + question
 - 6 Mistral AI generates a grounded answer using the document context
 - 7 The answer is returned to the user and saved to Buffer Memory
-